
LLaVA-UHD v4: What Makes Efficient Visual Encoding in MLLMs?

Kechen Fang¹ Yihua Qin¹ Chongyi Wang² Wenshuo Ma² Tianyu Yu¹ Yuan Yao^{1*}

¹Tsinghua University ²ModelBest

Abstract

Visual encoding constitutes a major computational bottleneck in Multimodal Large Language Models (MLLMs), especially for high-resolution image inputs. The prevailing practice typically adopts global encoding followed by post-ViT compression. Global encoding produces massive token sequences, while post-ViT compression incurs the full quadratic attention cost of the ViT before any token reduction takes place. In this work, we revisit this convention along two dimensions: the encoding strategy and visual token compression. First, controlled experiments show that slice-based encoding outperforms global encoding across benchmarks, suggesting that preserving local details through sliced views can be more beneficial than applying global attention for fine-grained perception. Second, we introduce intra-ViT early compression, which reduces tokens in shallow ViT layers and substantially lowers visual-encoding FLOPs while preserving downstream performance. By integrating intra-ViT compression into the slice-based encoding framework, we present LLaVA-UHD v4, an efficient and compute-controllable visual encoding scheme tailored for high-resolution inputs. Across a diverse set of benchmarks covering document understanding, OCR, and general VQA, LLaVA-UHD v4 reduces visual-encoding FLOPs by 55.8% while matching or even surpassing baseline performance. These results suggest that visual-encoding efficiency can be substantially improved without sacrificing downstream performance, providing a practical design direction for efficient high-resolution MLLMs. All model weights and code will be publicly released to support further research¹.

1 Introduction

Multimodal Large Language Models (MLLMs) have made remarkable progress on a broad spectrum of vision-language tasks [25, 20, 47, 2]. As the field shifts toward fine-grained perception [31, 33, 30] and detailed image understanding [52, 43], high-resolution image inputs are rapidly becoming the default. To preserve as much visual detail as possible and sustain downstream performance, the prevailing recipe is global encoding [41, 38], which feeds the full image directly into the vision encoder. As resolution grows, this yields a token sequence that scales with image area. To relieve the downstream LLM from this token explosion, mainstream frameworks then attach a compression module after the vision encoder [47]. That is, visual tokens are reduced only after the vision encoder has already executed full global self-attention at quadratic complexity. This approach is straightforward to implement, yet its computational cost increases rapidly with resolution. Furthermore, post-ViT compression cannot mitigate the ViT’s cost, as it only operates after the full computation has already occurred. This cost is far from negligible in the high-resolution regime, making high-resolution visual encoding a central efficiency bottleneck in modern MLLMs.

*Corresponding author

¹Code available at <https://github.com/THUMAI-Lab/LLaVA-UHD-v4>

In this work, we systematically rethink this inefficient convention, beginning with the encoding paradigm. The community has widely held that global encoding is the more direct and lossless choice, since it supplies complete global context and allows arbitrary patch-to-patch interaction [41, 38]. However, our empirical evaluations across diverse benchmarks yield a surprising conclusion that slice-based encoding consistently outperforms global encoding, suggesting that slice-based strategies can already provide sufficiently informative feature representations. Moreover, by processing large images via partitioning, slice-based encoding structurally sidesteps the quadratic blow-up incurred by global encoding, making it the more efficient paradigm for ultra-high-resolution images.

While slice-based encoding alleviates the per-forward attention explosion to some extent, high resolution still inherently produces a large number of tokens. Existing compression schemes, such as MLP-based spatial merging [41, 28], Pixel-Shuffle and various resamplers [20, 1] and token-pruning approaches [3], are almost exclusively post-ViT. They only ease the burden on the downstream LLM and do nothing about the heavy cost inside the encoder itself. To achieve truly extreme efficiency, we must strike at the root of the bottleneck: the ViT’s own compute. Intuitively, token compression must be moved inside the vision encoder and triggered as early as possible, so that the vast majority of ViT layers operate on only a small number of tokens. The vision encoder is typically a pretrained model, and inserting a randomly initialized compressor into its intermediate layers can perturb or even destroy its learned visual representations. Such modifications incur substantial additional training cost and offer no guarantee of recovering the original performance, making early in-ViT token compression a problem that demands careful design.

To address the challenges above, we introduce a parameter-reuse early compressor: a window-attention block coupled with a downsampling MLP, both inserted into the shallow layers of the ViT and initialized by reusing the pretrained weights of their adjacent ViT layers. This warm start places the new module very close to the representation manifold of the original ViT from the very first training step, thereby avoiding any disruption to the learned visual representations. The module compresses the ViT’s tokens by $4\times$ at a very early stage of the encoder, so that the vast majority of subsequent ViT layers operate on only a small fraction of the original token budget.

Combining slice-based encoding with the proposed intra-ViT early compression, we obtain LLaVA-UHD v4, an efficient and compute-controllable visual encoding architecture for high-resolution MLLMs. Across eight standard benchmarks, LLaVA-UHD v4 matches or surpasses a post-ViT baseline at the same $16\times$ compression ratio in overall downstream accuracy.

Our main contributions are as follows: (1) We revisit the common practice of global encoding and demonstrate the advantages of slice-based encoding in preserving fine-grained details while circumventing the quadratic computational overhead. (2) Building on this insight, we identify the limitations of post-ViT token compression and propose a novel intra-ViT shallow-layer compression architecture that directly addresses the computational bottleneck of visual encoding. (3) Integrating these two designs, we propose LLaVA-UHD v4, which combines slice-based encoding with an early compressor and maintains competitive performance while achieving a 55.75% acceleration in visual encoding FLOPs.

2 Rethinking High-Resolution Visual Encoding

We begin with a controlled study of two design choices that are central to high-resolution MLLMs: (1) How high-resolution images are encoded before entering the ViT. (2) How visual tokens are compressed along the pipeline. For both questions, we default to SigLIP 2 [40] as the ViT backbone and Qwen3 [46] as the LLM, while fixing the training data and the total visual-token budget reaching the LLM, so that any observed difference is attributable solely to the dimension under study.

2.1 Slice-based Encoding Outperforms Global Encoding

The community has converged on global encoding (GE) as the actual choice for high-resolution MLLMs [41, 28], on the intuitive grounds that feeding the full image to the ViT preserves complete global context and permits arbitrary patch-to-patch interaction. Slice-based encoding (SE) [14, 8], which partitions the image into smaller views encoded independently, is typically framed as a computational compromise, which sacrifices global context for tractable per-forward cost. In this

Table 1: **Comparison of encoding strategies.** We compare the two encoding strategies under different compression rates and data scales using SigLIP 2 as the ViT backbone. GE denotes global encoding and SE denotes slice-based encoding.

Data Scale	Method	MMMU	MathVista	MMB _{EN}	MMB _{CN}	MMStar	HallBench	AI2D	OCRBench	Avg.
Compression Rate 4×										
4M	GE	58.4	67.4	83.7	81.5	63.5	48.5	80.3	77.6	70.1
	SE	61.9	66.7	82.9	79.5	62.3	49.1	80.5	82.0	70.6
8M	GE	60.4	71.4	84.4	83.5	65.4	49.3	82.5	80.0	72.1
	SE	60.3	71.2	85.2	83.4	64.3	56.3	82.0	83.6	73.3
Compression Rate 16×										
4M	GE	58.4	62.7	80.3	81.9	60.4	47.7	78.5	72.0	67.7
	SE	57.9	63.0	79.4	79.1	60.6	50.5	77.7	77.5	68.2
8M	GE	58.7	65.6	82.9	82.6	60.5	47.0	80.0	73.6	68.9
	SE	58.6	67.3	83.7	82.3	62.9	51.2	79.8	79.1	70.6

section we test this framing directly: under matched compression and training conditions, which paradigm actually delivers better downstream accuracy?

Setup. The two paradigms share the ViT backbone, projector, LLM, and the post-ViT compressor, differing only in how the image is presented to the ViT. GE rescales the image to at most $N \times 448^2$ pixels and processes it in a single forward pass. SE decomposes the image into a thumbnail and a set of slices laid out by an aspect-ratio-aware best-grid policy. We sweep two compression ratios (4×, 16×) and two data scales (4M, 8M), and evaluate on the eight benchmarks. To comprehensively assess model performance, we conduct evaluations on a broad benchmark suite encompassing mathematics, OCR, and general VQA tasks.

SE consistently outperforms GE, with larger gains at higher scales. Table 1 reports the SigLIP-2-based comparison. Across all four settings, SE outperforms GE on average, with gains ranging from 0.5 to 1.7 points. The advantage also tends to increase with data scale, growing from 0.5 to 1.2 points under 4× compression and from 0.5 to 1.7 points under 16× compression. In the SigLIP-2 sweep, the SE margin increases from 4M to 8M under both compression ratios, suggesting that the observed benefit persists with additional supervision in this setting. In particular, the advantage is most pronounced on OCR-intensive tasks requiring fine-grained recognition, where SE leads GE by 3.6 to 5.5 points on OCRBench across the four SigLIP-2 settings.

Robustness. To ensure that the observed advantage of SE is not attributable to a specific backbone or slicing configuration, we conduct two stress tests under more demanding conditions, with average accuracy reported in Table 2. First, we replace SigLIP 2 with MoonViT [39, 38], a ViT explicitly pretrained on native-resolution inputs, where SE retains an average margin of approximately +1.5 points across both 8M and 16M data scales, indicating that its effectiveness generalizes across visual encoders. Second, under the 16×/8M setting, we adopt an alternative slicing schedule with a fourfold larger slice budget, which preserves higher per-image resolution and exposes the encoder to substantially more high-resolution visual tokens. Under this more demanding slicing configuration, the margin further widens to more than +2 points on average, with substantially larger gains on OCR-intensive tasks. Taken together, these results suggest that, under the resolution settings considered, the benefit of SE increases with input resolution and exhibits no evidence of saturation. Per-benchmark results for both stress tests are provided in Table A1.

Table 2: **Robustness of slice-based encoding.** Average accuracy under (i) an alternative vision encoder backbone and (ii) a higher-resolution slicing schedule, both at compression rate 16×.

Setting	Scale	GE	SE
MoonViT	8M	70.3	71.6
	16M	72.2	73.6
Higher-Res	8M	68.8	71.0

***Finding 1.** Slice-based encoding consistently matches or outperforms global encoding across different compression rates, vision encoder backbones, and image resolutions.*

Analysis. Across different backbones and slicing schedules, slice-based encoding (SE) consistently matches or outperforms global encoding (GE). We attribute this to a difference in inductive bias: SE

preserves locality by decomposing the image into spatially coherent views, allowing the encoder to focus its capacity on fine-grained patterns within each slice, whereas GE processes the entire image jointly, forcing local details to compete with global context under a fixed token budget. A more detailed analysis is provided in Appendix B.1.

2.2 Compressing Visual Tokens at High Resolution

Slice-based encoding (Section 2.1) provides a stronger input pipeline, yet each high-resolution image still produces a large number of visual tokens that must be compressed before entering the LLM. These are conventionally compressed by a connector module placed between the ViT and the LLM. We address two questions about this scheme. First, which connector design performs best? Second, is this post-ViT compression sufficient enough at high resolution?

Setup. Two families dominate the connector designs. Query-based resamplers [2, 1, 20] attend a small set of learnable queries to the ViT output via cross-attention. Spatial-merging MLPs [23, 8] fold neighboring patch tokens via pixel-unshuffle and project them through a lightweight feed-forward network. We first compare both under matched conditions, sharing the ViT backbone, LLM, training recipe, slice-based encoding, and target token counts at $4\times$ and $16\times$ compression. Both are evaluated on the eight benchmarks of Section 2.1 across multiple data scales.

Table 3: **Connector comparison.**

Downsampling	Scale	Resampler	MLP
$4\times$	4M	65.51	69.10
	8M	64.80	71.73
$16\times$	4M	65.87	66.64
	8M	67.66	68.84
	16M	70.39	70.81

MLP outperforms resampler. Table 3 reports the comparison results. The MLP connector outscores the resampler across all configurations, with the largest margins at lower compression ratios where it leads by +3.3 to +6.7 points at $4\times$. We further observe that the gap narrows as the compression ratio tightens and training data scales up, falling to +0.4 points at $16\times$ compression with 16M training data, though MLP retains its lead in every cell.

***Finding 2.** Pixel-unshuffle-based MLP downsampling provides a stronger post-ViT compression baseline than query-based resampler.*

Analysis. Pixel-unshuffle strictly preserves spatial structure by mapping each $k \times k$ ViT patch group into one token with concatenated channels, maintaining a coarse 2D layout. In contrast, the resampler uses content-agnostic learnable queries with global attention, discarding explicit spatial correspondence. The decisive factor is therefore not capacity (the resampler in fact uses more parameters at lower compression yet still loses by the largest margins) but whether spatial priors are built-in or must be learned. A more detailed analysis is provided in Appendix B.2.

Together, *Findings 1 and 2* establish slice-based encoding combined with an MLP connector as an effective baseline. However, because this token reduction occurs only after the vision encoder, it merely relieves the downstream LLM while leaving the ViT’s massive internal compute entirely unchanged. To overcome this structural bottleneck, compression must be shifted inside the ViT pipeline. We detail the structure of our proposed intra-ViT compressor in Section 3.

3 LLaVA-UHD v4

In this section, we answer the design questions raised at the end of Section 2.2 and introduce LLaVA-UHD v4. It builds on the slice-based encoding and MLP connector established in Section 2 and adds an intra-ViT early compressor $\mathcal{D}$. We describe the end-to-end architecture in Section 3.1, and introduce the design principles, structure, and parameter-reuse initialization in Section 3.2.

3.1 Overview

Figure 1 shows the full pipeline. Following *Finding 1*, the input image is decomposed into a low-resolution thumbnail and a small set of high-resolution slices selected by an aspect-ratio-aware policy.

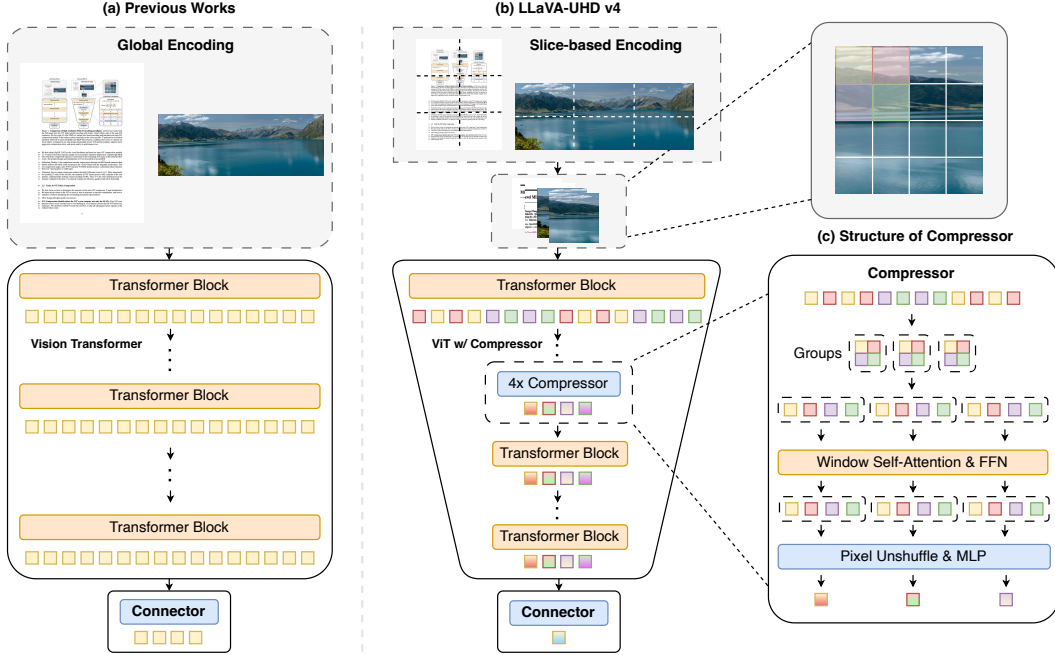


Figure 1: **Comparison of high-resolution MLLM encoding paradigms.** (a) Previous works feed the full image into the ViT under global encoding and reduce visual tokens only at the post-ViT connector. (b) Our work, LLaVA-UHD v4, adopts slice-based encoding and introduces an intra-ViT compression module $\mathcal{D}$ that reduces token count early in the vision encoder. $\mathcal{D}$ performs local window attention followed by pixel unshuffle and MLP-based fusion, enabling later layers to operate on fewer tokens. Compared to (a), this design substantially lowers ViT-internal compute, supports more aggressive compression ratios, and incurs nearly no performance loss.

All views are rescaled and concatenated along the sequence dimension, and processed in a single ViT forward pass that preserves per-view attention locality.

We then adopt SigLIP 2 [40] as the visual backbone and insert an intra-ViT compression module $\mathcal{D}$. $\mathcal{D}$ reduces the token sequence length via local window-attention followed by a lightweight MLP, after which the compressed sequence is processed by the remaining ViT layers at the reduced token count. The detailed design and initialization of $\mathcal{D}$ are described in Section 3.2.

Following *Finding 2*, the compressed encoder output passes through an MLP-based connector that further reduces the token count and projects the visual features into the language model space. The two compression stages, intra-ViT $\mathcal{D}$ and post-ViT MLP, jointly produce a substantial token reduction from raw visual patches to LLM input.

Ultimately, this two-stage compression reduces the final LLM token count to $\frac{1}{16}N$. More importantly, by inserting $\mathcal{D}$ early in the encoder, the majority of ViT layers process only a quarter of the raw patches, fundamentally slashing visual-encoding FLOPs. Since $\mathcal{D}$ is the only modification to the baseline validated in Section 2, we directly evaluate its efficiency-quality trade-off in Section 4.

3.2 Early In-ViT Token Compression

We first focus on determining the structure and initialization of the intra-ViT compressor $\mathcal{D}$. We must decide where in the ViT to insert it, how to structure its internal computation, and how to initialize it without disrupting the surrounding pretrained representation.

Three design principles guide our answers.

(P1) Compression should reduce the ViT’s own compute, not only the LLM’s. Post-ViT compression leaves every encoder layer’s cost unchanged, as all tokens traverse the full ViT before any

reduction. We therefore embed $\mathcal{D}$ inside the encoder, so that all subsequent layers operate at the reduced token count.

(P2) The compressor should sit as early as possible, balanced against representational depth. Earlier insertion maximizes savings, while deeper placement retains more pretrained processing at full resolution and better aligns with the downstream representation manifold. Our ablations (Section 4.3) identify $k=6$ as the best efficiency-quality trade-off.

(P3) Inserting $\mathcal{D}$ must not disrupt the pretrained representation manifold. A pretrained ViT is tightly calibrated, with each layer expecting the distribution produced by its predecessor. A randomly initialized $\mathcal{D}$ would perturb this distribution and turn fine-tuning into the harder problem of recovering the pretrained manifold from scratch. We therefore initialize $\mathcal{D}$ by reusing the parameters of the preceding ViT layer (Section 3.2.2), so that fine-tuning begins on the manifold rather than searching for it.

Together, these three principles fix $\mathcal{D}$'s placement and initialization strategy. It remains to specify the internal computation of $\mathcal{D}$ and the precise weight-inheritance mechanism, which we address in the rest of this section.

3.2.1 Window-Attention Downsampling Module

The pretrained ViT consists of L transformer layers operating on token sequences $\mathbf{X}_l \in \mathbb{R}^{N \times d}$. We insert a downsampling module $\mathcal{D}$ between layers k and $k+1$. The module takes $\mathbf{X}_k$ as input and produces a compressed sequence $\tilde{\mathbf{X}} \in \mathbb{R}^{N/4 \times d}$, after which the remaining layers operate at the reduced token resolution. The module $\mathcal{D}$ consists of two conceptual steps: (i) a window-attention block that enriches local context, and (ii) a downsample-and-fuse block that reduces spatial resolution while aggregating information.

Window attention. We first apply a window attention operator $\text{WinAttn}_{2 \times 2}$ on $\mathbf{X}_k$, producing an intermediate representation $\mathbf{Y}$. The attention is restricted to non-overlapping 2×2 windows, so each token interacts only with its three spatial neighbors. This design ensures that tokens exchange information exactly within the region that will be merged in the next step.

Downsample and fuse. A 2×2 PixelUnshuffle operation directly reshapes $\mathbf{Y}$ into $\mathbf{Z} \in \mathbb{R}^{N/4 \times 4d}$. An MLP then fuses these concatenated channels back to dimension d , yielding the final output $\tilde{\mathbf{X}}$.

This design cleanly separates local context aggregation from information-preserving downsampling and channel fusion, while keeping the module compatible with the pretrained ViT stack.

3.2.2 Parameter-Reuse Initialization

The downsampling module $\mathcal{D}$ introduces three parameterized components: the window-attention sub-block, the fused MLP ($\mathbf{W}_1, \phi, \mathbf{W}_2$), and the two LayerNorms. A standard random initialization would inject substantial noise into the encoder's intermediate representations. In practice, this perturbation lengthens fine-tuning and is not guaranteed to recover the pretrained ViT's effective representation manifold at all.

We instead initialize $\mathcal{D}$ entirely from the weights of the pretrained ViT layer k that immediately precedes it. This parameter reuse serves two purposes: it eliminates randomly-initialized parameters from the encoder's compute path entirely, and, as we make precise below, it places $\mathcal{D}$ at $t = 0$ in close functional correspondence to a surrogate operation derived from layer k itself, so that fine-tuning starts on or near the pretrained representation manifold. We initialize $\mathcal{D}$ as follows:

Window attention. The attention projections, head configuration, and LN_1 are copied directly from layer k . The only modification is the 2×2 window mask, which restricts attention to local neighborhoods while preserving the original attention weights.

Fused MLP. We construct the MLP to mimic applying the FFN of layer k independently to each of the four patches within a 2×2 window, followed by averaging. Concretely,

$$\mathbf{W}_1 = \text{BlockDiag}(\mathbf{F}_1^{(k)}, \mathbf{F}_1^{(k)}, \mathbf{F}_1^{(k)}, \mathbf{F}_1^{(k)}), \quad \mathbf{W}_2 = \frac{1}{4}[\mathbf{F}_2^{(k)} \mid \mathbf{F}_2^{(k)} \mid \mathbf{F}_2^{(k)} \mid \mathbf{F}_2^{(k)}].$$

The bias follows the original FFN and is not scaled, so that the fused output corresponds to averaging four FFN branches while preserving the bias magnitude.

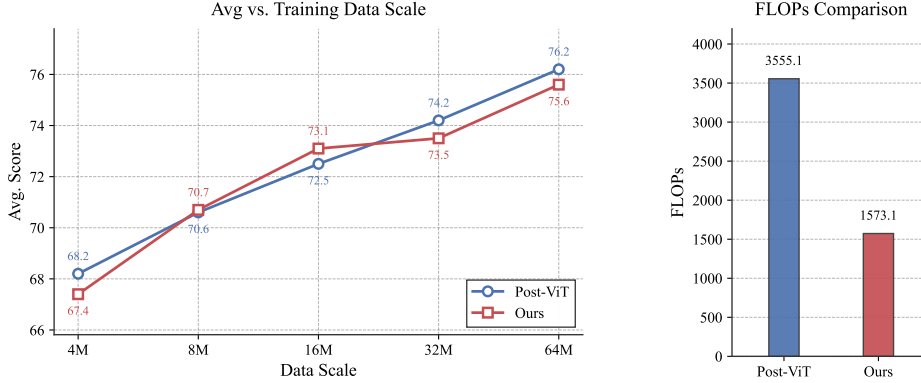


Figure 2: **Average performance and computational cost.** Left: average accuracy across training data scales, comparing LLaVA-UHD v4 and the post-ViT baseline. Right: FLOPs comparison between the two systems.

LayerNorm and residual. LN_2 is applied over the concatenated $4d$ features with tiled affine parameters, and the residual branch is implemented as a parameter-free 2×2 average pooling.

4 Experiment

We empirically validate the design of LLaVA-UHD v4 through controlled comparisons against the best-performing configuration from the pilot study (slice-based encoding with a $16\times$ post-ViT MLP compressor, hereafter the *post-ViT baseline*). Section 4.1 describes the setup, Section 4.2 reports the main quality-efficiency results across training data scales, and Section 4.3 analyzes the key design choices of the intra-ViT compressor.

4.1 Experimental Setup

Architecture. Unless otherwise stated, LLaVA-UHD v4 uses SigLIP 2 [40] as the vision encoder and Qwen3-8B [46] as the language model. The intra-ViT compression module $\mathcal{D}$ is inserted after layer $k = 6$ and reduces the per-slice token count by $4\times$. A post-ViT MLP compressor further downsamples by $4\times$, yielding an end-to-end $16\times$ reduction. Unless otherwise stated, the FLOPs are computed for processing a single slice through the ViT, i.e., the visual-encoding cost per input slice.

Training. We follow a four-stage recipe: (i) *Vision-language alignment* on large-scale image-text pairs, updating only the projector and $\mathcal{D}$; (ii) *Knowledge injection* via OCR, document, and chart data with only ViT unfrozen; (iii) *Interleaved training* on image-text sequences for multi-image and long-context reasoning; and (iv) *Supervised instruction tuning* on a diverse mixture of general VQA, math, and conversational tasks. Detailed hyperparameters are in Appendix C.

Benchmarks. We evaluate on eight benchmarks covering three capability dimensions: (i) *general VQA*: MMBench_{EN} [26], MMBench_{CN} [26], MMStar [7]; (ii) *knowledge & reasoning*: MMMU [50], MathVista [29], AI2D [19], HallusionBench [13]; (iii) *fine-grained perception*: OCRBench [27].

4.2 Main Results

Intra-ViT early compression matches the post-ViT baseline in accuracy while substantially reducing visual-encoding cost. As shown in Figure 2 and Figure 3, we compare LLaVA-UHD v4 against the strongest post-ViT baseline under identical training settings and a shared end-to-end $16\times$ compression ratio. By shifting a $4\times$ compression stage inside the ViT, all subsequent layers operate on only 25% of the original tokens. This structurally reduces visual-encoding FLOPs from 3555G to 1573G, a massive 55.75% reduction. Despite this aggressive early compression, LLaVA-UHD v4 performs within ± 0.8 points of the baseline across all five training scales, with a negligible mean deviation of only -0.29 points. This demonstrates that our intra-ViT design yields massive compute savings without compromising downstream accuracy.

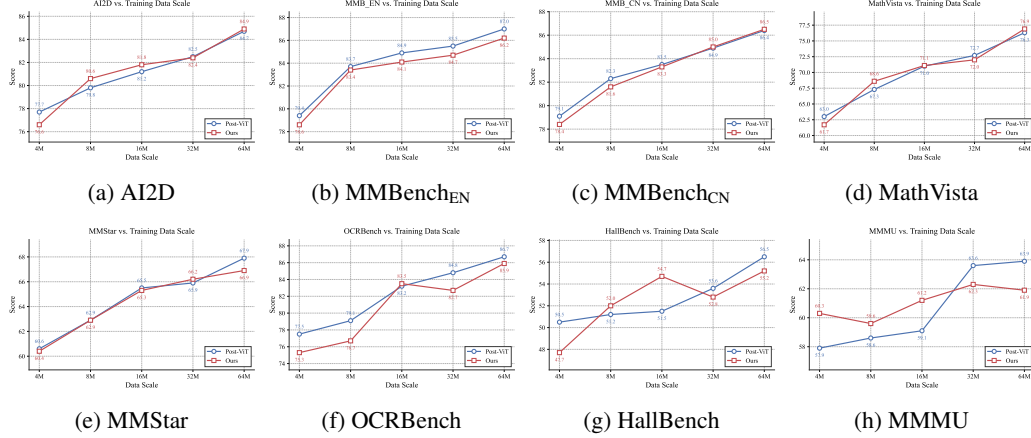


Figure 3: **Benchmark trends across training data scales.** We compare Post-ViT and our method on eight benchmarks across different training data scales.

The proposed early-compression design preserves average scaling behavior within the tested range. As training data increases from 4M to 64M samples, both systems improve substantially. The post-ViT baseline rises from 68.2 to 76.2 average points, while LLaVA-UHD v4 rises from 67.4 to 75.6. The average gap stays within ± 0.8 points and does not widen monotonically, suggesting that intra-ViT compression does not introduce an observable average-level scaling ceiling. Individual benchmarks still show scale-dependent variation, for example, MMMU favors LLaVA-UHD v4 at smaller scales but the post-ViT baseline at larger scales, but this reversal does not indicate a systematic compression failure, since the aggregate trend remains stable across the tested range.

4.3 Ablations on the In-ViT Compression Module

Section 4.2 shows that LLaVA-UHD v4 can match the post-ViT baseline under the same final token budget. We now ablate the design of the intra-ViT compression module $\mathcal{D}$ to understand why this is possible. All variants use the 8M in-house training set and an end-to-end $16\times$ compression ratio, with $\mathcal{D}$ inserted at $k = 6$ by default, applying $4\times$ reduction over 2×2 token windows. *Average Pool* and *Pixel-Unshuffle* are parameter-free or randomly initialized merging baselines. *Cross-Attn* collapses each window into one token via cross-attention with either the top-left or mean query. *Win-Attn* variants first apply window self-attention and then fuse tokens with a Pixel-Unshuffle MLP, either randomly initialized or reused from the preceding ViT FFN. The central question is therefore not whether early compression can reduce compute, but which compressor can preserve the pretrained ViT representation while doing so.

Table 4: **Ablations on in-ViT compression designs.** All variants use the same final $16\times$ compression ratio and insertion depth $k = 6$.

(a) Naive merging			(b) Direct cross-attention			(c) Reused MLP and window attention		
Method	FLOPs (G)	Avg.	Method	FLOPs (G)	Avg.	Method	FLOPs (G)	Avg.
Post-ViT Base	3555.1	70.6	Post-ViT Base	3555.1	70.6	Pix-Unshuffle	1401.2	69.8
Avg Pool	1368.7	69.6	Cross (top-left)	1402.0	70.5	Reused MLP	1490.2	69.9
Pix-Unshuffle	1401.2	69.8	Cross (mean)	1402.0	69.9	Win w/ MLP	1484.1	70.1
						Win w/ Reused	1573.1	70.7

Naive in-ViT compression is efficient but not sufficient. Table 4(a) first evaluates simple in-ViT merging strategies. Moving compression into the ViT substantially reduces computation, from 3555.1G FLOPs for the post-ViT baseline to 1401.2G FLOPs for in-ViT variants. However, this efficiency gain does not automatically recover baseline-level accuracy. Average pooling is the cheapest design, but drops the average score from 70.6 to 69.6. A learnable pixel-unshuffle MLP improves the score to 69.8, but still remains below the post-ViT baseline. These results suggest that early token reduction creates a nontrivial interface problem within the pretrained ViT, requiring the

compressor to reduce sequence length while maintaining compatibility with the representational distribution expected by the remaining encoder layers.

Window attention and reuse initialization are complementary components of the structured merger. Table 4(c) factorizes our structured merger along two axes, whether local window attention is applied before merging, and whether the fusion MLP is initialized by reusing the preceding ViT FFN weights. Reuse alone brings only a marginal gain over a randomly initialized pixel-unshuffle MLP, improving the average score from 69.8 to 69.9. Window attention alone is more helpful, raising the score to 70.1. When the two are combined, the score reaches 70.7, exceeding both individual modifications and slightly surpassing the post-ViT baseline. The gain is super-additive because the two components together make the merger closely resemble a standard vision encoder block, with local self-attention followed by an FFN and both initialized from the preceding layer’s weights. The output of the merger therefore stays close to what the subsequent ViT layers were pretrained to consume. Neither component alone achieves this alignment. Without window attention, the reused MLP is applied to tokens that have not been locally contextualized as in pretraining, so its initialization provides little benefit. Without reuse, window attention restores local structure but the randomly initialized fusion then maps the contextualized tokens out of the pretrained input distribution.

Direct cross-attention merging underperforms local window attention followed by a reuse-initialized MLP.

Table 4(b) compares against a more direct alternative that collapses each 2×2 window into a single token through local cross-attention. This alternative is competitive when the top-left token is used as the query, reaching 70.5 average accuracy, close to both the post-ViT baseline and our final design. However, changing the query to the window mean lowers the score to 69.8 under the same FLOPs, showing that direct one-step aggregation is sensitive to how the representative query is constructed. In contrast, first updating all tokens through local window attention and then fusing the contextualized tokens with a reuse-initialized MLP achieves 70.7, the best among all ablated in-ViT compressors. As shown in Table A6, this query sensitivity persists at 16M, where the better-performing query even flips to the window mean, while Win-Attn with Reused MLP stays strongest at both scales. This suggests a structural issue rather than a tuning artifact, since no single query consistently captures what a 2×2 window should be summarized into, whereas updating all tokens before fusion sidesteps the question entirely.

Effective intra-ViT compression requires an intermediate insertion depth. As shown in Table 5, inserting $\mathcal{D}$ too early is highly destructive: $k = 3$ gives the lowest FLOPs, but drops the average score to 38.76. This indicates that the earliest ViT layers have not yet formed representations that are safe to merge. In contrast, inserting at $k = 6$ preserves accuracy while retaining most of the compute savings. Delaying compression to $k = 9$ or $k = 15$ brings no accuracy benefit, yielding slightly lower scores while increasing FLOPs to 1901G and 2557G, respectively. Among the non-collapsed settings in our sweep, $k = 6$ is therefore Pareto-favorable. It is both more accurate and more efficient than the deeper insertion depths. This suggests that effective intra-ViT compression requires an intermediate depth where tokens are no longer purely low-level visual features but have already accumulated enough semantic structure to be safely merged.

Table 5: **Effect of insertion depth k on accuracy and compute.** Evaluation for $\mathcal{D}$ inserted after different ViT layers, reporting average score and visual-encoding FLOPs.

Layer (k)	FLOPs (G)	Avg. Score
3	1245.1	39.7
6	1573.1	70.7
9	1901.1	70.3
15	2557.0	70.4

5 Conclusion

In this work, we present LLaVA-UHD v4, a highly efficient visual encoding architecture that systematically re-examines high-resolution perception in MLLMs. By demonstrating the empirical advantages of slice-based encoding over the global encoding paradigm, and introducing a novel parameter-reusing intra-ViT early compression module, we substantially reduce the severe computational bottleneck inside the vision encoder. Extensive experiments validate that our approach reduces visual-encoding FLOPs by 55.75% under a $16\times$ compression ratio, while matching or surpassing the fine-grained downstream performance of strong post-ViT baselines. While our current module operates at a fixed compression rate, exploring dynamic, content-aware token reduction mechanisms within the encoder remains an exciting direction for future research. Together, these results suggest that aggressive token

reduction can be performed inside the vision encoder without sacrificing fine-grained perception, offering a practical path toward more scalable multimodal foundation models.

References

- [1] Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katherine Millican, Malcolm Reynolds, et al. Flamingo: a visual language model for few-shot learning. *Advances in neural information processing systems*, 35:23716–23736, 2022.
- [2] Jinze Bai, Shuai Bai, Shusheng Yang, Shijie Wang, Sinan Tan, Peng Wang, Junyang Lin, Chang Zhou, and Jingren Zhou. Qwen-vl: A versatile vision-language model for understanding, localization, text reading, and beyond, 2023.
- [3] Daniel Bolya, Cheng-Yang Fu, Xiaoliang Dai, Peizhao Zhang, Christoph Feichtenhofer, and Judy Hoffman. Token merging: Your vit but faster. *arXiv preprint arXiv:2210.09461*, 2022.
- [4] Junbum Cha, Wooyoung Kang, Jonghwan Mun, and Byungseok Roh. Honeybee: Locality-enhanced projector for multimodal llm. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 13817–13827, 2024.
- [5] Liang Chen, Haozhe Zhao, Tianyu Liu, Shuai Bai, Junyang Lin, Chang Zhou, and Baobao Chang. An image is worth 1/2 tokens after layer 2: Plug-and-play inference acceleration for large vision-language models. In *European Conference on Computer Vision*, pages 19–35. Springer, 2024.
- [6] Lin Chen, Jinsong Li, Xiaoyi Dong, Pan Zhang, Conghui He, Jiaqi Wang, Feng Zhao, and Dahua Lin. Sharegpt4v: Improving large multi-modal models with better captions. In *European Conference on Computer Vision*, pages 370–387. Springer, 2024.
- [7] Lin Chen, Jinsong Li, Xiaoyi Dong, Pan Zhang, Yuhang Zang, Zehui Chen, Haodong Duan, Jiaqi Wang, Yu Qiao, Dahua Lin, et al. Are we on the right way for evaluating large vision-language models? *Advances in Neural Information Processing Systems*, 37:27056–27087, 2024.
- [8] Zhe Chen, Weiyun Wang, Hao Tian, Shenglong Ye, Zhangwei Gao, Erfei Cui, Wenwen Tong, Kongzhi Hu, Jiapeng Luo, Zheng Ma, et al. How far are we to gpt-4v? closing the gap to commercial multimodal models with open-source suites. *Science China Information Sciences*, 67(12):220101, 2024.
- [9] Zhe Chen, Jiannan Wu, Wenhai Wang, Weijie Su, Guo Chen, Sen Xing, Muyan Zhong, Qinglong Zhang, Xizhou Zhu, Lewei Lu, et al. Internvl: Scaling up vision foundation models and aligning for generic visual-linguistic tasks. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 24185–24198, 2024.
- [10] Wenliang Dai, Junnan Li, Dongxu Li, Anthony Tiong, Junqi Zhao, Weisheng Wang, Boyang Li, Pascale N Fung, and Steven Hoi. Instructblip: Towards general-purpose vision-language models with instruction tuning. *Advances in neural information processing systems*, 36:49250–49267, 2023.
- [11] Mostafa Dehghani, Basil Mustafa, Josip Djolonga, Jonathan Heek, Matthias Minderer, Mathilde Caron, Andreas Steiner, Joan Puigcerver, Robert Geirhos, Ibrahim M Alabdulmohsin, et al. Patch n’pack: Navit, a vision transformer for any aspect ratio and resolution. *Advances in Neural Information Processing Systems*, 36:2252–2274, 2023.
- [12] Enrico Fini, Mustafa Shukor, Xiujun Li, Philipp Dufter, Michal Klein, David Haldimann, Sai Aitharaju, Victor G Turrissi da Costa, Louis Béthune, Zhe Gan, et al. Multimodal autoregressive pre-training of large vision encoders. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 9641–9654, 2025.
- [13] Tianrui Guan, Fuxiao Liu, Xiyang Wu, Ruiqi Xian, Zongxia Li, Xiaoyu Liu, Xijun Wang, Lichang Chen, Furong Huang, Yaser Yacoob, et al. Hallusionbench: an advanced diagnostic suite for entangled language hallucination and visual illusion in large vision-language models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 14375–14385, 2024.
- [14] Zonghao Guo, Ruyi Xu, Yuan Yao, Junbo Cui, Zanlin Ni, Chunjiang Ge, Tat-Seng Chua, Zhiyuan Liu, and Gao Huang. Llava-uhd: an lmm perceiving any aspect ratio and high-resolution images. In *European Conference on Computer Vision*, pages 390–406. Springer, 2024.
- [15] Anwen Hu, Haiyang Xu, Jiabo Ye, Ming Yan, Liang Zhang, Bo Zhang, Ji Zhang, Qin Jin, Fei Huang, and Jingren Zhou. mplug-docowl 1.5: Unified structure learning for ocr-free document understanding. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 3096–3120, 2024.

- [16] Shaohan Huang, Li Dong, Wenhui Wang, Yaru Hao, Saksham Singhal, Shuming Ma, Tengchao Lv, Lei Cui, Owais Khan Mohammed, Barun Patra, et al. Language is not all you need: Aligning perception with language models. *Advances in Neural Information Processing Systems*, 36:72096–72109, 2023.
- [17] Gabriel Ilharco, Mitchell Wortsman, Nicholas Carlini, Rohan Taori, Achal Dave, Vaishaal Shankar, Hongseok Namkoong, John Miller, Hannaneh Hajishirzi, Ali Farhadi, et al. Openclip. *Zenodo*, 2021.
- [18] Siddharth Karamcheti, Suraj Nair, Ashwin Balakrishna, Percy Liang, Thomas Kollar, and Dorsa Sadigh. Prismatic VLMs: Investigating the design space of visually-conditioned language models. In *International Conference on Machine Learning (ICML)*, 2024.
- [19] Aniruddha Kembhavi, Mike Salvato, Eric Kolve, Minjoon Seo, Hannaneh Hajishirzi, and Ali Farhadi. A diagram is worth a dozen images. In *European conference on computer vision*, pages 235–251. Springer, 2016.
- [20] Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. Blip-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *International conference on machine learning*, pages 19730–19742. PMLR, 2023.
- [21] Yanwei Li, Yuechen Zhang, Chengyao Wang, Zhisheng Zhong, Yixin Chen, Ruihang Chu, Shaoteng Liu, and Jiaya Jia. Mini-gemini: Mining the potential of multi-modality vision language models. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2025.
- [22] Zhihang Lin, Mingbao Lin, Luxi Lin, and Rongrong Ji. Boosting multimodal large language models with visual tokens withdrawal for rapid inference. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 5334–5342, 2025.
- [23] Haotian Liu, Chunyuan Li, Yuheng Li, and Yong Jae Lee. Improved baselines with visual instruction tuning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 26296–26306, 2024.
- [24] Haotian Liu, Chunyuan Li, Yuheng Li, Bo Li, Yuanhan Zhang, Sheng Shen, and Yong Jae Lee. Llava-next: Improved reasoning, ocr, and world knowledge, January 2024.
- [25] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. *Advances in neural information processing systems*, 36:34892–34916, 2023.
- [26] Yuan Liu, Haodong Duan, Yuanhan Zhang, Bo Li, Songyang Zhang, Wangbo Zhao, Yike Yuan, Jiaqi Wang, Conghui He, Ziwei Liu, et al. Mmbench: Is your multi-modal model an all-around player? In *European conference on computer vision*, pages 216–233. Springer, 2024.
- [27] Yuliang Liu, Zhang Li, Mingxin Huang, Biao Yang, Wenwen Yu, Chunyuan Li, Xu-Cheng Yin, Cheng-Lin Liu, Lianwen Jin, and Xiang Bai. Ocrbench: on the hidden mystery of ocr in large multimodal models. *Science China Information Sciences*, 67(12):220102, 2024.
- [28] Dongchen Lu, Yuyao Sun, Zilu Zhang, Leping Huang, Jianliang Zeng, Mao Shu, and Huo Cao. Internvl-x: Advancing and accelerating internvl series with efficient visual token compression. *arXiv preprint arXiv:2503.21307*, 2025.
- [29] Pan Lu, Hritik Bansal, Tony Xia, Jiacheng Liu, Chunyuan Li, Hannaneh Hajishirzi, Hao Cheng, Kai-Wei Chang, Michel Galley, and Jianfeng Gao. Mathvista: Evaluating mathematical reasoning of foundation models in visual contexts. *arXiv preprint arXiv:2310.02255*, 2023.
- [30] Ahmed Masry, Xuan Long Do, Jia Qing Tan, Shafiq Joty, and Enamul Hoque. Chartqa: A benchmark for question answering about charts with visual and logical reasoning. In *Findings of the association for computational linguistics: ACL 2022*, pages 2263–2279, 2022.
- [31] Minesh Mathew, Dimosthenis Karatzas, and C.V. Jawahar. DocVQA: A Dataset for VQA on Document Images. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 2200–2209, 2021.
- [32] Brandon McKinzie, Zhe Gan, Jean-Philippe Fauconnier, Sam Dodge, Bowen Zhang, Philipp Dufter, Dhruvi Shah, Xianzhi Du, Futang Peng, Anton Belyi, et al. Mm1: methods, analysis and insights from multimodal llm pre-training. In *European Conference on Computer Vision*, pages 304–323. Springer, 2024.
- [33] Linke Ouyang, Yuan Qu, Hongbin Zhou, Jiawei Zhu, Rui Zhang, Qunshu Lin, Bin Wang, Zhiyuan Zhao, Man Jiang, Xiaomeng Zhao, et al. Omnidocbench: Benchmarking diverse pdf document parsing with comprehensive annotations. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 24838–24848, 2025.

- [34] Zhiliang Peng, Wenhui Wang, Li Dong, Yaru Hao, Shaohan Huang, Shuming Ma, and Furu Wei. Kosmos-2: Grounding multimodal large language models to the world. *arXiv preprint arXiv:2306.14824*, 2023.
- [35] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PmLR, 2021.
- [36] Yongming Rao, Wenliang Zhao, Benlin Liu, Jiwen Lu, Jie Zhou, and Cho-Jui Hsieh. Dynamicvit: Efficient vision transformers with dynamic token sparsification. *Advances in neural information processing systems*, 34:13937–13949, 2021.
- [37] Quan Sun, Yuxin Fang, Ledell Wu, Xinlong Wang, and Yue Cao. Eva-clip: Improved training techniques for clip at scale. *arXiv preprint arXiv:2303.15389*, 2023.
- [38] Kimi Team, Tongtong Bai, Yifan Bai, Yiping Bao, SH Cai, Yuan Cao, Y Charles, HS Che, Cheng Chen, Guanduo Chen, et al. Kimi k2. 5: Visual agentic intelligence. *arXiv preprint arXiv:2602.02276*, 2026.
- [39] Kimi Team, Angang Du, Bohong Yin, Bowei Xing, Bowen Qu, Bowen Wang, Cheng Chen, Chenlin Zhang, Chenzhuang Du, Chu Wei, et al. Kimi-vl technical report. *arXiv preprint arXiv:2504.07491*, 2025.
- [40] Michael Tschannen, Alexey Gritsenko, Xiao Wang, Muhammad Ferjad Naeem, Ibrahim Alabdulmohsin, Nikhil Parthasarathy, Talfan Evans, Lucas Beyer, Ye Xia, Basil Mustafa, et al. Siglip 2: Multilingual vision-language encoders with improved semantic understanding, localization, and dense features. *arXiv preprint arXiv:2502.14786*, 2025.
- [41] Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, et al. Qwen2-vl: Enhancing vision-language model’s perception of the world at any resolution. *arXiv preprint arXiv:2409.12191*, 2024.
- [42] Weihan Wang, Qingsong Lv, Wenmeng Yu, Wenyi Hong, Ji Qi, Yan Wang, Junhui Ji, Zhuoyi Yang, Lei Zhao, Xixuan Song, et al. Cogvlm: Visual expert for pretrained language models. *Advances in Neural Information Processing Systems*, 37:121475–121499, 2024.
- [43] Penghao Wu and Saining Xie. V?: Guided visual search as a core mechanism in multimodal llms. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 13084–13094, 2024.
- [44] Long Xing, Qidong Huang, Xiaoyi Dong, Jiajie Lu, Pan Zhang, Yuhang Zang, Yuhang Cao, Conghui He, Jiaqi Wang, Feng Wu, et al. Pyramiddrop: Accelerating your large vision-language models via pyramid visual redundancy reduction. *arXiv preprint arXiv:2410.17247*, 2024.
- [45] Hu Xu, Saining Xie, Xiaoqing Ellen Tan, Po-Yao Huang, Russell Howes, Vasu Sharma, Shang-Wen Li, Gargi Ghosh, Luke Zettlemoyer, and Christoph Feichtenhofer. Demystifying clip data. *arXiv preprint arXiv:2309.16671*, 2023.
- [46] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [47] Yuan Yao, Tianyu Yu, Ao Zhang, Chongyi Wang, Junbo Cui, Hongji Zhu, Tianchi Cai, Haoyu Li, Weilin Zhao, Zhihui He, et al. Minicpm-v: A gpt-4v level mllm on your phone. *arXiv preprint arXiv:2408.01800*, 2024.
- [48] Qinghao Ye, Haiyang Xu, Guohai Xu, Jiabo Ye, Ming Yan, Yiyang Zhou, Junyang Wang, Anwen Hu, Pengcheng Shi, Yaya Shi, et al. mplug-owl: Modularization empowers large language models with multimodality. *arXiv preprint arXiv:2304.14178*, 2023.
- [49] Hongxu Yin, Arash Vahdat, Jose M Alvarez, Arun Mallya, Jan Kautz, and Pavlo Molchanov. A-vit: Adaptive tokens for efficient vision transformer. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10809–10818, 2022.
- [50] Xiang Yue, Yuansheng Ni, Kai Zhang, Tianyu Zheng, Ruoyi Liu, Ge Zhang, Samuel Stevens, Dongfu Jiang, Weiming Ren, Yuxuan Sun, et al. Mmmu: A massive multi-discipline multimodal understanding and reasoning benchmark for expert agi. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 9556–9567, 2024.
- [51] Xiaohua Zhai, Basil Mustafa, Alexander Kolesnikov, and Lucas Beyer. Sigmoid loss for language image pre-training. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 11975–11986, 2023.

- [52] Yi-Fan Zhang, Huanyu Zhang, Haochen Tian, Chaoyou Fu, Shuangqing Zhang, Junfei Wu, Feng Li, Kun Wang, Qingsong Wen, Zhang Zhang, et al. Mme-realworld: Could your multimodal llm challenge high-resolution real-world scenarios that are difficult for humans? *arXiv preprint arXiv:2408.13257*, 2024.
- [53] Yuan Zhang, Chun-Kai Fan, Junpeng Ma, Wenzhao Zheng, Tao Huang, Kuan Cheng, Denis Gudovskiy, Tomoyuki Okuno, Yohei Nakata, Kurt Keutzer, et al. Sparsevlm: Visual token sparsification for efficient vision-language model inference. *arXiv preprint arXiv:2410.04417*, 2024.
- [54] Deyao Zhu, Jun Chen, Xiaoqian Shen, Xiang Li, and Mohamed Elhoseiny. Minigpt-4: Enhancing vision-language understanding with advanced large language models. *arXiv preprint arXiv:2304.10592*, 2023.

A Related Work

A.1 Vision Encoder

Language-supervised contrastive models remain the dominant choice for MLLMs due to their natural pre-alignment with language. CLIP [35] and its variants [51, 17, 45, 37] have progressively refined this paradigm through improved objectives, data curation and parameter scale. More recently, SigLIP 2 [40] unifies contrastive, captioning, self-distillation and masked prediction objectives into a single recipe, achieving broad improvements in classification, localization, and MLLM transfer. Despite their dominance, these encoders inherit a language bottleneck: they capture only what alt-text describes and exhibit "CLIP-blind" failures on fine-grained spatial distinctions, and most operate at fixed low resolutions. To push beyond these limits, a parallel line scales the visual backbone itself, exemplified by InternViT-6B [9] and AIMv2 [12], while NaViT [11] and the native-resolution ViTs of Qwen2-VL [41], Kimi K2.5 [38] make token count scale with image area. Another major line keeps the encoder fixed and instead partitions high-resolution inputs into multiple low-resolution slices that are encoded independently, as in LLaVA-NeXT [24], Intern VL 1.5 [8], LLaVA-UHD [14] and mPLUG-DocOwl 1.5 [15]. While effective at preserving fine-grained details with off-the-shelf encoders, slicing multiplies visual tokens and fragments cross-slice spatial context. However, scaling the encoder to billions of parameters or to native high resolutions inflates visual token counts and pretraining cost, creating a tension between visual fidelity and MLLMs efficiency.

A.2 Multimodal Connector

Bridging a vision encoder to an LLM requires a connector module, and the field has converged on two dominant designs. Query-based resamplers were popularized by Flamingo [1]’s Perceiver Resampler, which compresses arbitrary spatio-temporal feature grids to a fixed 64 latent tokens via cross-attention with learned queries, and by BLIP-2 [20]’s Q-Former, a 32-query bottleneck transformer pretrained with contrastive matching and generative objectives. This recipe was widely inherited: MiniGPT-4 [54] freezes BLIP-2’s Q-Former and trains only a linear head, InstructBLIP [10] makes the queries instruction-aware, Qwen-VL [2] employs a single-layer cross-attention compressor producing 256 tokens. Kosmos-1/2 [16, 34] and mPLUG-Owl [48] all adopt Perceiver- or abstractor-style pooling, primarily for token-count efficiency. Projection-based connectors offer a competing minimalist alternative: LLaVA [25]’s single linear layer and LLaVA-1.5 [23]’s two-layer GELU MLP retain every patch token, showing that simple token-preserving projection can match or exceed resamplers trained on orders of magnitude more data, and this design has since been widely adopted by many subsequent MLLMs [24, 6, 21, 18, 42]. Yet the empirical record is contradictory: Honeybee [4] attribute large gains to locality-preserving projection, whereas MM1 [32] finds connector architecture nearly negligible relative to image resolution and visual-token count. This leaves the trade-off between information fidelity and token efficiency unresolved and motivating a direct empirical comparison between Resampler- and MLP-style connectors.

A.3 Token Compression

The hundreds to thousands of visual tokens produced make token compression a central concern for MLLM efficiency. Existing approaches operate at three points of the pipeline. Inside the LLM, a line of largely training-free methods prunes visual tokens between transformer layers, exploiting the observation that visual tokens become increasingly redundant at deeper layers. FastV [5] drops low-attention visual tokens after an early layer, while SparseVLM [53], VTW [22], and PyramidDrop [44] extend this idea with text-aware or progressive schedules. Such methods are simple to deploy but inherit whatever redundancy the encoder has already produced. Between the encoder and the LLM, a learnable compressor distills patch tokens before they enter the language model. Inside the ViT, compression directly reduces the cost of the visual backbone itself. ToMe [3] bipartite-matches and merges similar tokens at each layer without retraining; DynamicViT [36] and A-ViT [49] learn to drop uninformative tokens during the forward pass. In-encoder compression accelerates the entire backbone, but is tightly coupled to the encoder’s pretraining objective and risks discarding tokens that downstream language grounding would have relied on.

Table A1: **Detailed results for the robustness study of slice-based encoding.** Detailed breakdown of Table 2 across the eight benchmarks, covering both the MoonViT backbone and the higher-resolution slicing schedule under compression rate $16\times$.

Data Scale	Method	MMMU	MathVista	MMB _{EN}	MMB _{CN}	MMStar	HallBench	AI2D	OCRBench	Avg.
MoonViT (Compression Rate $16\times$)										
8M	GE	57.8	69.0	82.9	82.2	61.3	50.7	80.1	78.0	70.3
	SE	58.8	70.1	82.7	82.2	64.4	52.0	80.1	82.2	71.6
16M	GE	57.7	73.4	83.8	82.6	65.3	53.3	82.7	79.0	72.2
	SE	62.4	72.2	83.6	82.9	66.3	54.1	81.8	85.1	73.6
Higher-Resolution (Compression Rate $16\times$)										
8M	GE	56.4	66.2	82.6	82.0	61.1	48.4	79.7	73.9	68.8
	SE	59.1	68.4	84.4	83.3	62.4	49.9	79.1	81.5	71.0

Table A2: **Main comparison on in-house data across training scales.** Both systems share an identical architecture, training recipe, data, and end-to-end $16\times$ compression ratio; they differ only in where compression occurs. Avg. is computed over the eight benchmarks shown. *Post-ViT baseline* performs all compression after the ViT. *Ours* performs $4\times$ compression inside the ViT after layer 6 and another $4\times$ after the ViT.

Data Scale	Method	MMMU	MathVista	MMB _{EN}	MMB _{CN}	MMStar	HallBench	AI2D	OCRBench	Avg.
4M	Post-ViT	57.9	63.0	79.4	79.1	60.6	50.5	77.7	77.5	68.2
	Ours	60.3	61.7	78.6	78.4	60.4	47.7	76.6	75.3	67.4
8M	Post-ViT	58.6	67.3	83.7	82.3	62.9	51.2	79.8	79.1	70.6
	Ours	59.6	68.6	83.4	81.6	62.9	52.0	80.6	76.7	70.7
16M	Post-ViT	59.1	71.0	84.9	83.5	65.5	51.5	81.2	83.2	72.5
	Ours	61.2	71.1	84.1	83.3	65.3	54.7	81.8	83.5	73.1
32M	Post-ViT	63.6	72.7	85.5	84.9	65.9	53.6	82.5	84.8	74.2
	Ours	62.3	72.0	84.7	85.0	66.2	52.8	82.4	82.7	73.5
64M	Post-ViT	63.9	76.3	87.0	86.4	67.9	56.5	84.7	86.7	76.2
	Ours	61.9	76.9	86.2	86.5	66.9	55.2	84.9	85.9	75.6

B Detailed Analysis and Results

B.1 Detailed Analysis of Encoding Strategies

Across the evaluated SigLIP 2 settings, MoonViT settings, and slicing schedules, slice-based encoding (SE) improves the average score over global encoding (GE), although individual benchmark outcomes remain mixed. The MoonViT comparison shows that this average advantage persists even with a backbone designed for native-resolution processing, and the higher-resolution slicing variant further suggests that the result is not tied to a single slicing budget. We therefore interpret SE not merely as a computational workaround, but as an encoding strategy that changes the context in which visual features are formed before compression.

The key difference lies less in the compression ratio itself than in the attention context used by the ViT. With the pixel-unshuffle MLP compressor, both GE and SE apply a locality-preserving spatial merge, so the compressor does not globally pool all visual tokens. However, the features entering this compressor have been produced under different encoding contexts. GE encodes the full image in a single ViT forward pass, where all patches interact in one global attention space. SE decomposes the image into a thumbnail and spatially coherent slices, then encodes each slice independently, so the ViT forms features within localized views before those features are spatially merged.

This local encoding bias is especially relevant for fine-grained perception. GE preserves unrestricted patch-to-patch interaction inside the ViT, which is useful for global context but may dilute the inductive bias toward local structure. SE sacrifices some within-ViT global interaction, yet it encourages the visual encoder to extract text, chart marks, and dense document patterns within local neighborhoods before the same type of spatial compression is applied. The largest and most stable gains on OCRBench are consistent with this interpretation: tasks that depend heavily on small local structures appear to benefit from forming visual features in localized views before compression.

Table A3: **Full per-benchmark results for in-ViT compression design ablations.** All variants share the same end-to-end $16\times$ compression ratio and insertion depth $k = 6$, differing only in how the $4\times$ in-ViT compression stage is realized. FLOPs are reported per slice through the ViT, and bold marks the best score in each column.

Method	FLOPs (G)	MMMU	MathVista	MMB _{EN}	MMB _{CN}	MMStar	HallBench	AI2D	OCRBench	Avg.
<i>Post-ViT merging</i>										
Post-ViT Baseline	3555.1	58.6	67.3	83.7	82.3	62.9	51.2	79.8	79.1	70.6
<i>Naive in-ViT merging</i>										
Average Pool	1368.7	59.2	67.2	83.6	81.5	62.4	47.1	79.8	75.7	69.6
Pixel-Unshuffle MLP	1401.2	58.7	66.7	82.4	81.4	61.6	49.2	80.0	78.6	69.8
Reused MLP	1490.2	57.6	67.0	81.8	81.3	62.3	48.8	81.0	79.5	69.9
<i>Cross-attention merging</i>										
Cross-Attn (top-left query)	1402.0	59.9	68.6	83.6	81.5	61.1	50.8	80.1	78.2	70.5
Cross-Attn (mean query)	1402.0	61.0	66.0	82.2	81.5	61.5	47.5	80.6	78.5	69.9
<i>Window-attention merging</i>										
Win-Attn w/ MLP	1484.1	58.8	67.4	83.5	81.7	62.7	47.3	80.5	78.9	70.1
Win-Attn w/ Reused MLP	1573.1	59.6	68.6	83.4	81.6	62.9	52.0	80.6	76.7	70.7

Table A4: **Comparison of connector designs.** We compare the MLP downsampler against the resampler under the SE setting across multiple downsampling rates. OCRBench is divided by 10, and Avg. is computed over the eight benchmarks shown.

Data Scale	Connector	MMMU	MathVista	MMB _{EN}	MMB _{CN}	MMStar	HallBench	AI2D	OCRBench	Avg.
<i>Downsampling Rate $4\times$</i>										
4M	Resampler	57.4	62.7	80.3	78.9	60.7	46.2	78.1	73.9	67.3
	MLP	61.9	66.7	82.9	79.5	62.3	49.1	80.5	82.0	70.6
8M	Resampler	57.9	61.7	80.4	77.9	58.9	49.1	78.2	68.7	66.6
	MLP	60.3	71.2	85.2	83.4	64.3	56.3	82.0	83.6	73.3
<i>Downsampling Rate $16\times$</i>										
4M	Resampler	58.7	62.3	79.6	78.2	59.7	49.5	76.9	75.1	67.5
	MLP	57.9	63.0	79.4	79.1	60.6	50.5	77.7	77.5	68.2
8M	Resampler	57.1	65.9	81.9	81.3	61.3	49.1	80.3	78.3	69.4
	MLP	58.6	67.3	83.7	82.3	62.9	51.2	79.8	79.1	70.6
16M	Resampler	59.1	69.1	84.0	83.5	64.1	54.3	81.2	81.2	72.1
	MLP	59.1	71.0	84.9	83.5	65.5	51.5	81.2	83.2	72.5

Within our tested settings, the advantage of SE therefore appears to come less from the compressor itself and more from the locality of the preceding visual encoding.

B.2 Detailed Results of Connector Designs

The detailed results in Table A4 clarify why we use the MLP connector as the post-ViT baseline. Its largest gains appear at $4\times$ compression, where the output sequence still preserves a relatively rich coarse layout. In this regime, pixel-unshuffle can exploit its built-in spatial structure: each output token is formed from a fixed local patch group and remains tied to a local image neighborhood. The resampler, by contrast, summarizes the ViT output through learnable queries, so its outputs no longer have fixed spatial correspondence and must learn this organization from data.

As compression becomes more aggressive, the gap narrows but does not reverse. At $16\times$ compression, both connectors must discard more spatial detail, reducing the benefit of an explicitly locality-preserving merge. Even in the most favorable setting for the resampler, with 16M training samples, MLP remains slightly ahead. This suggests that the resampler can partially learn useful aggregation with enough data and a tight token budget, but it does not provide a stronger default than the simpler spatially structured connector. We therefore use the MLP connector as the strongest post-ViT baseline before asking whether part of the compression should be moved inside the ViT.

Table A5: **Ablation on the open-source LLaVA-OneVision setting.** We evaluate different in-ViT compressor designs under the open-source dataset.

Method	MMMU	MathVista	MMB _{EN}	MMB _{CN}	MMStar	HallBench	AI2D	OCRBench	Avg.
LLaVA-OneVision Open-source Setting									
Post-ViT Baseline	46.3	62.2	74.9	71.6	56.7	40.3	79.9	64.7	62.1
Average Pool	47.6	62.4	75.4	73.1	56.3	40.3	81.5	62.9	62.4
Pixel-Unshuffle MLP	46.6	62.3	72.8	72.2	51.7	38.3	80.2	58.7	60.4
Reused MLP	45.3	60.4	76.1	74.1	55.3	40.7	81.2	63.7	62.1
Cross-Attn (top-left)	48.6	62.0	75.1	72.5	56.4	44.7	80.5	64.8	63.1
Cross-Attn (mean)	47.6	62.4	75.4	73.1	56.3	40.3	81.5	62.9	62.4
Win-Attn w/ MLP	50.9	61.4	75.4	73.9	54.7	42.7	81.8	65.0	63.2
Win-Attn w/ Reused MLP	48.3	63.5	76.7	73.5	57.0	42.7	81.1	64.6	63.4

Table A6: **Comparison of different ViT internal downsampling strategies across training scales.** All systems share an identical architecture, training recipe, data, and end-to-end $16\times$ compression ratio. They differ only in the downsampling module design.

Data Scale	Method	MMMU	MathVista	MMB _{EN}	MMB _{CN}	MMStar	HallBench	AI2D	OCRBench	Avg.
8M	Win-Attn w/ Reused MLP	59.6	68.6	83.4	81.6	62.9	52.0	80.6	76.7	70.7
	Cross-Attn (top-left)	59.9	68.6	83.6	81.5	61.1	50.8	80.1	78.2	70.5
	Cross-Attn (mean)	61.0	66.0	82.2	81.5	61.5	47.5	80.6	78.5	69.8
16M	Win-Attn w/ Reused MLP	61.2	71.1	84.1	83.7	65.3	54.7	81.8	83.5	73.1
	Cross-Attn (top-left)	61.2	69.2	85.2	83.7	63.5	52.6	82.3	81.0	72.3
	Cross-Attn (mean)	61.0	69.3	84.6	83.1	64.4	55.3	81.4	83.2	72.8

B.3 Additional Ablations on the Open-Source LLaVA-OneVision Setting

Table A5 further evaluates the same family of in-ViT downsampling designs under the open-source LLaVA-OneVision training setting. The trend is broadly consistent with the in-house ablations in the main paper: naively inserting a learnable MLP merger inside the ViT is not sufficient, as the plain MLP variant drops from the baseline average of 62.1 to 60.4. In contrast, designs that introduce local interaction before token reduction are substantially more robust. Cross-attention and window-attention variants improve over the plain MLP, suggesting that early compression benefits from first allowing the tokens within each local 2×2 region to exchange information.

Among all variants, *Win-Attn w/ Reused MLP* achieves the best average score, improving the baseline from 62.1 to 63.4. The gain is modest but consistent with the main-paper conclusion: local contextualization and parameter-reuse initialization are complementary. Compared with *Win-Attn w/ MLP*, reuse improves the average score from 63.2 to 63.4. This mixed per-benchmark pattern indicates that the open-source setting is somewhat noisier, but the best average performance still comes from the reused window-attention design, supporting its transfer beyond the in-house training recipe.

C Hyperparameters

Table A7 and Table A8 provide the detailed optimization settings for the four-stage training recipe described in Section 4.1. Both recipes begin with a warmup stage for vision-language alignment, continue with high-quality image training, and end with supervised instruction tuning. The tables report the learning-rate schedule, training length, warmup steps, trainable modules, and packing-equivalent per-GPU batch size for the in-house data setting and the LLaVA-OneVision training setting, respectively.

D Limitations

While LLaVA-UHD v4 significantly accelerates high-resolution visual encoding, several limitations remain for future work. First, our intra-ViT compression module applies a fixed and uniform spatial downsampling rate across all patches. It does not adapt to the varying information density within an image, making dynamic, content-aware token reduction (e.g., allocating more tokens to dense text and fewer to plain backgrounds) an important next step. Second, the optimal insertion depth for the compressor ($k = 6$) was empirically determined for the SigLIP 2 backbone; migrating

Table A7: Training hyperparameters on in-house data.

Stage	LR	LR _{min}	Trainable	Batch size
1	1.0×10^{-4}	5.0×10^{-5}	ViT / Connector	32
2	1.0×10^{-5}	5.0×10^{-6}	ViT	6
3	5.0×10^{-5}	1.0×10^{-5}	Full	6
4	1.0×10^{-5}	1.0×10^{-6}	Full	9

Table A8: Training hyperparameters on LLaVA-OneVision data.

Stage	LR	LR _{min}	Trainable	Batch size
1	1.0×10^{-4}	5.0×10^{-5}	ViT / Connector	16
2	1.0×10^{-5}	5.0×10^{-6}	Full	20
3	5.0×10^{-5}	1.0×10^{-5}	Full	34
4	1.0×10^{-5}	1.0×10^{-6}	Full	11

to architecturally distinct or substantially deeper vision encoders may require re-evaluating this hyperparameter. Finally, although slice-based encoding excels at fine-grained perception, it inherently fragments high-resolution context across slice boundaries, relying primarily on the low-resolution thumbnail to bridge global interactions.